

PABNA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering and Technology

Department of Information and Communication Engineering
(ICE)

PRACTICAL LAB REPORT

COURSE INFORMATION

Course Code: ICE-4206

Course Title: Neural Network Sessional

SUBMITTED BY

Md. Nazmul Hossain Emon

Roll No.: 200621

Session: 2019-20

Year / Sem: 4th Year, 2nd Semester

Department: Information &
Communication Engineering

University: Pabna University of Science
and Technology

SUBMITTED TO

Dr. Md. Imran Hosain

Designation: Associate Professor

Department: Information &
Communication Engineering

University: Pabna University of Science
and Technology

Location: Pabna, Bangladesh

Date of Submission: 16/05/2026

Exp. No.	Experiment Name	Page No.
01	Write a MATLAB or Python program using perceptron net for AND function with bipolar inputs and targets. The convergence curves and the decision boundary lines are also shown.	01
02	Implement the SGD Method using Delta learning rule for following input-target sets. X_Input = [0 0 1; 0 1 1; 1 0 1; 1 1 1], D_Target = [0; 0; 1; 1]	04
03	Implement the Batch Method using Delta learning rule for following input-target sets. X_Input = [0 0 1; 0 1 1; 1 0 1; 1 1 1], D_Target = [0; 0; 1; 1]	07
04	Compare the performance of SGD and the Batch method using the delta learning rule.	09
05	Write a MATLAB or Python program to recognize the image of digits. The input images are five-by-five pixel squares, which display five numbers from 1 to 5, as shown in Figure 1.	12
06	Write a MATLAB or Python program to classify face/fruit/bird using Convolution Neural Network (CNN).	15
07	Consider an Artificial Neural Network (ANN) with three layers. Write a MATLAB or Python program to learn this network using Back Propagation Network.	18
08	Write a MATLAB or Python program to recognize the numbers 1 to 4 from speech signal using Artificial Neural Network (ANN).	22
09	Use a pretrained ResNet-50 model for fine-tuning on a custom dataset (e.g., flower classification).	24
10	Train a GAN to generate synthetic handwritten digits using MNIST.	27

Experiment No: 01

Experiment Name: Write a MATLAB or Python program using perceptron net for AND function with bipolar inputs and targets. The convergence curves and the decision boundary lines are also shown.

Objectives

1. To implement a perceptron learning algorithm in MATLAB or Python for the AND logic function using bipolar inputs and targets.
2. To visualize the convergence curve (error vs. epoch) to observe how the perceptron learns over time.
3. To plot the decision boundary line that separates the two classes after training is complete.

Theory

A Perceptron is the simplest form of an artificial neural network, introduced by Frank Rosenblatt in 1958. It is a single-layer supervised learning algorithm used for binary classification tasks. The perceptron takes a set of input signals, multiplies each by a corresponding weight, sums them up, and passes the result through an activation function to produce an output.

Bipolar Representation uses values of +1 and -1 (instead of 1 and 0 used in binary representation). For the AND function with bipolar inputs:

x₁	x₂	Target (d)
+1	+1	+1
+1	-1	-1
-1	+1	-1
-1	-1	-1

The Perceptron Learning Rule updates weights according to:

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \alpha \cdot (\mathbf{d} - \hat{\mathbf{y}}) \cdot \mathbf{x}$$

where:

α = learning rate

$\mathbf{d}$ = desired/target output

$\hat{\mathbf{y}}$ = actual output of the perceptron

$\mathbf{x}$ = input vector

The activation function used is the bipolar step function:

- Output = +1, if net input ≥ 0
- Output = -1, if net input < 0

Training continues epoch by epoch until the network converges, i.e., there is zero error across all training samples. The convergence curve plots the total error against the number of epochs, showing the learning progress. The decision boundary is a straight line defined by:

$$w_1x_1 + w_2x_2 + b = 0$$

This line separates the input space into two regions corresponding to the two output classes (+1 and -1).

Code (Python):

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

X = np.array([[ -1, -1], [-1, 1], [1, -1], [1, 1]])
T = np.array([-1, -1, -1, 1])

w = np.zeros(2)
b = 0.0
eta = 0.1
errors = []

for epoch in range(20):
    err = 0
    for xi, ti in zip(X, T):
        y = np.sign(np.dot(w, xi) + b)
        if y == 0:
            y = -1
        if y != ti:
            w += eta * ti * xi
            b += eta * ti
            err += 1
    errors.append(err)
    if err == 0:
        break

print("Final weights:", w)
print("Final bias:", b)

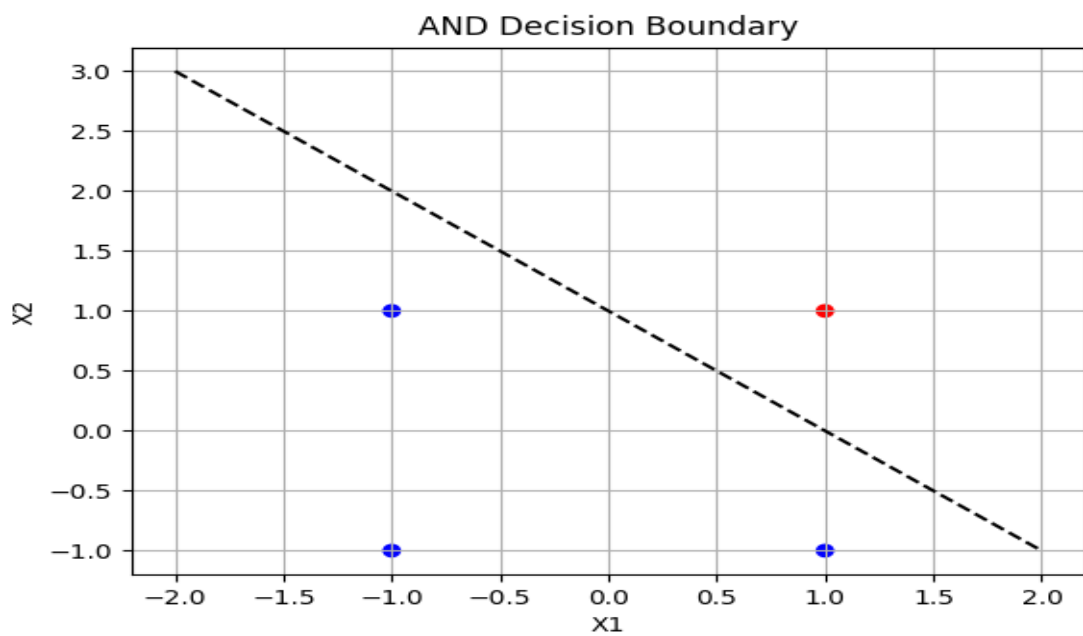
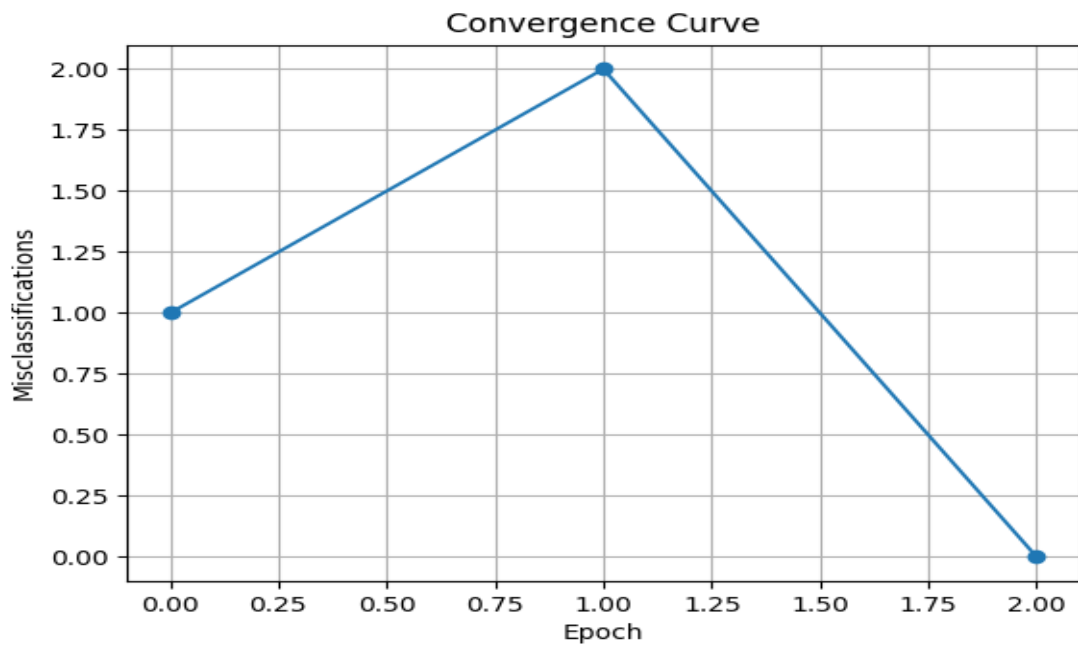
plt.plot(errors, marker='o')
plt.title("Convergence")
plt.xlabel("Epoch")
plt.ylabel("Misclassifications")
plt.grid(True)
plt.show()

xs = np.linspace(-2, 2, 100)
ys = (-w[0]*xs - b) / w[1]

plt.scatter(X[:,0], X[:,1], c=T, cmap='bwr')
plt.plot(xs, ys, 'k--')
plt.title("AND Decision Boundary")
plt.grid(True)
plt.show()
```

Output:

Final weights: [0.1 0.1]
Final bias: -0.1



Experiment No: 02

Experiment Name: Implement the SGD Method using Delta learning rule for following input-target sets. $X_{Input} = [0\ 0\ 1; 0\ 1\ 1; 1\ 0\ 1; 1\ 1\ 1]$, $D_{Target} = [0; 0; 1; 1]$

Objectives:

1. To implement the Stochastic Gradient Descent (SGD) method using the Delta learning rule for a given input-target dataset in MATLAB or Python.
2. To observe how the network weights are updated sample-by-sample and analyze the convergence behavior through error vs. epoch plots.
3. To evaluate the trained network's output and verify it against the desired targets.

Theory:

The Delta Learning Rule (also known as the Widrow-Hoff rule) is a supervised learning algorithm used to train single-layer neural networks. It minimizes the mean squared error (MSE) between the desired output and the actual output by adjusting the weights in the direction of the negative gradient of the error.

Stochastic Gradient Descent (SGD) updates the weights after each individual training sample, rather than waiting for all samples to be processed. This makes it faster to converge in many practical scenarios and is less computationally expensive per update.

Input and Target Sets:

x_1	x_2	Bias (x_3)	Target (d)
0	0	1	0
0	1	1	0
1	0	1	1
1	1	1	1

The net input to the neuron is calculated as:

$$net = w_1x_1 + w_2x_2 + w_3x_3$$

where $x_3 = 1$ acts as the bias term.

The activation function used is the linear (identity) function:

$$\hat{y} = net$$

The weight update rule using SGD with the Delta rule is:

$$w(new) = w(old) + \alpha \cdot (d - \hat{y}) \cdot x$$

where:

- α = learning rate
- $\mathbf{d}$ = desired/target output
- $\hat{\mathbf{y}}$ = actual output
- $\mathbf{x}$ = input vector (including bias)

The error for each sample is:

$$\mathbf{e} = \mathbf{d} - \hat{\mathbf{y}}$$

And the Mean Squared Error (MSE) over all samples is:

$$MSE = (1/n) \cdot \sum (\mathbf{d}_i - \hat{\mathbf{y}}_i)^2$$

In SGD, weight updates are applied after each sample, so within a single epoch the weights change multiple times. This is the key distinction from the Batch method. Training continues until the MSE falls below a defined threshold or a maximum number of epochs is reached. The convergence curve (MSE vs. epoch) shows how quickly the network learns the mapping from inputs to targets.

Code:

```
import numpy as np
np.random.seed(42)
X = np.array([[0,0,1], [0,1,1], [1,0,1], [1,1,1]], dtype=float)
D = np.array([0,0,1,1], dtype=float)
w = np.zeros(3); eta = 0.1; n_epochs = 100; mse_hist=[]
for epoch in range(n_epochs):
    for xi, di in zip(X,D):
        yi = np.dot(w, xi)
        w += eta * (di-yi) * xi
    y_all = X @ w
    mse_hist.append(np.mean((D - y_all)**2))
print("Trained weights:", np.round(w,4), "Predictions:", np.round(X @ w,4), "Targets:", D, f"Final MSE:{mse_hist[-1]:.6f}")

# 03 - Batch Delta
import numpy as np
np.random.seed(42)
X = np.array([[0,0,1],[0,1,1],[1,0,1],[1,1,1]], dtype=float)
D = np.array([0,0,1,1], dtype=float)
w = np.zeros(3); eta=0.5; n_epochs=100; mse_hist=[]
for epoch in range(n_epochs):
    Y = X @ w
    grad = (X.T @ (D - Y)) / len(X)
    w += eta * grad
    mse_hist.append(np.mean((D - Y)**2))
print("Trained weights:", np.round(w,4), "Predictions:", np.round(X @ w,4), "Targets:", D, f"Final MSE:{mse_hist[-1]:.6f}")
```

Output:

Trained weights: [9.997e-01 -3.000e-04 4.000e-04] Predictions : [4.0000e-04 1.0000e-04
1.0001e+00 9.9980e-01]

Targets: [0. 0. 1. 1.]

Final MSE : 0.000000

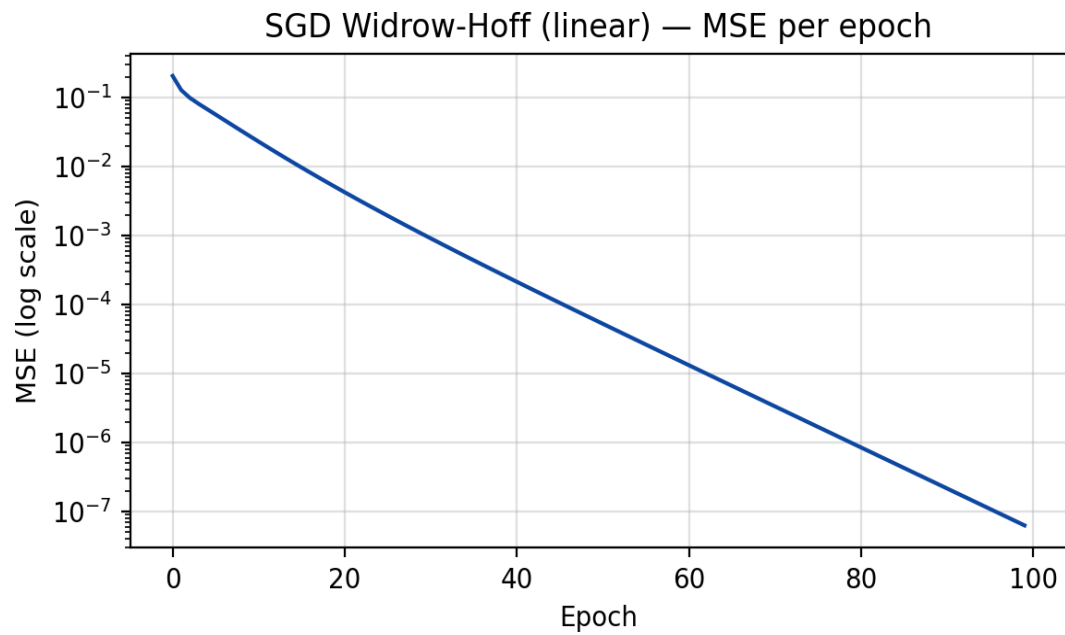


Figure: SGD Widrow – Hoff MSE per epoch on the question data.

Experiment No: 03

Experiment Name: Implement the Batch Method using Delta learning rule for following input-target sets. $X_{Input} = [0\ 0\ 1; 0\ 1\ 1; 1\ 0\ 1; 1\ 1\ 1]$, $D_{Target} = [0; 0; 1; 1]$

Objectives:

1. To implement the Batch Gradient Descent method using the Delta (Widrow-Hoff) learning rule for a given set of input-target pairs.
2. To observe how the Batch method accumulates errors over all training samples before updating weights, and to analyze its convergence behavior.
3. To compare the final learned weights and the convergence curve of the Batch method with that of the SGD method.

Theory:

The Delta Learning Rule, also known as the Widrow-Hoff or Least Mean Square (LMS) rule, is a supervised learning algorithm used to train single-layer neural networks. It minimizes the mean squared error (MSE) between the desired output and the actual output by adjusting weights in the direction of the negative gradient.

Batch Gradient Descent (also called Batch Learning) processes the entire training dataset before making a single weight update. Unlike Stochastic Gradient Descent (SGD), which updates weights after each individual sample, Batch method accumulates the total error (or total gradient) across all training samples and then performs one update per epoch.

The weight update rule for the Batch method is:

$$\Delta w = \alpha \cdot \sum (d^{(k)} - y^{(k)}) \cdot x^{(k)} \quad w(new) = w(old) + \Delta w$$

where the summation runs over all training patterns k .

Input-Target Set:

x_1	x_2	x_3 (bias)	Target (d)
0	0	1	0
0	1	1	0
1	0	1	1
1	1	1	1

Here, the third input $x_3 = 1$ acts as a bias input, allowing the network to shift the decision boundary.

The net input to the neuron is computed as:

$$net = w_1x_1 + w_2x_2 + w_3x_3$$

The activation function used is the linear (identity) function for computing error, while a threshold function may be used for final binary output:

- Output = 1, if $net \geq 0.5$
- Output = 0, if $net < 0.5$

Key characteristics of the Batch Method:

- Weight update is performed once per epoch after accumulating errors from all samples.
- The convergence is smoother compared to SGD since individual noisy updates are averaged out.
- It is computationally more stable but may converge slower than SGD for large datasets.
- The total error per epoch is used to track convergence: training stops when the error falls below a defined threshold.

Code:

```
import numpy as np
np.random.seed(42)
X = np.array([[0,0,1],[0,1,1],[1,0,1],[1,1,1]], dtype=float)
D = np.array([0,0,1,1], dtype=float)
w = np.zeros(3); eta=0.5; n_epochs=100; mse_hist=[]
for epoch in range(n_epochs):
    Y = X @ w
    grad = (X.T @ (D - Y)) / len(X)
    w += eta * grad
    mse_hist.append(np.mean((D - Y)**2))
print("Trained weights:", np.round(w,4), "Predictions:",
      np.round(X @ w,4), "Targets:", D, f"Final MSE:{mse_hist[-1]:.6f}")
```

Output:

```
Trained weights: [ 9.999e-01 -1.000e-04  1.000e-04]
Predictions: [1.000e-04 0.000e+00 1.000e+00 9.999e-01]
Targets: [0. 0. 1. 1.] Final MSE:0.000000
```

Experiment No: 04

Experiment Name: Compare the performance of SGD and the Batch method using the delta learning rule.

Objectives:

1. To implement both Stochastic Gradient Descent (SGD) and Batch learning methods using the Delta learning rule and compare their performance on the same dataset.
2. To analyze the convergence speed and error behavior of SGD vs. Batch method by plotting their respective error curves over epochs.
3. To determine which method converges faster and more stably for the given input-target dataset.

Theory:

The Delta Learning Rule (also known as the Widrow-Hoff or LMS rule) is a gradient descent-based supervised learning algorithm used to adjust the weights of a single-layer neural network. It minimizes the Mean Squared Error (MSE) between the desired output and the actual output. The weight update formula is:

$$\Delta w = \alpha \cdot (d - \hat{y}) \cdot x$$

where:

- α = learning rate
- d = desired/target output
- $\hat{y}$ = actual network output
- x = input vector

There are two primary methods of applying the Delta rule:

1. Stochastic Gradient Descent (SGD) Method:

In SGD, the weights are updated after every single training sample. For each input pattern presented to the network, the error is computed immediately and the weights are adjusted before moving to the next sample.

- **Weight update:** performed per sample
- **Convergence:** faster in early epochs but noisy
- **Error curve:** fluctuates due to per-sample updates

2. Batch Method:

In the Batch method, all training samples are presented to the network first, and the weight updates are accumulated over the entire dataset. The weights are updated only once per epoch using the total accumulated error.

- **Weight update:** performed once per epoch
- **Convergence:** smoother but may require more epochs
- **Error curve:** smooth and monotonically decreasing

Dataset used:

<i>X</i> Input ($x_1, x_2, bias$)	<i>D</i> Target
0, 0, 1	0
0, 1, 1	0
1, 0, 1	1
1, 1, 1	1

Performance Comparison:

Feature	SGD Method	Batch Method
Weight Update	After each sample	After each epoch
Convergence Speed	Faster initially	Slower but stable
Error Curve	Noisy/fluctuating	Smooth
Computational Cost	Lower per update	Higher per epoch
Sensitivity to LR	More sensitive	Less sensitive

The comparison of the two methods is typically visualized by plotting the error vs. epoch curves for both on the same graph, allowing direct observation of convergence behavior.

Code:

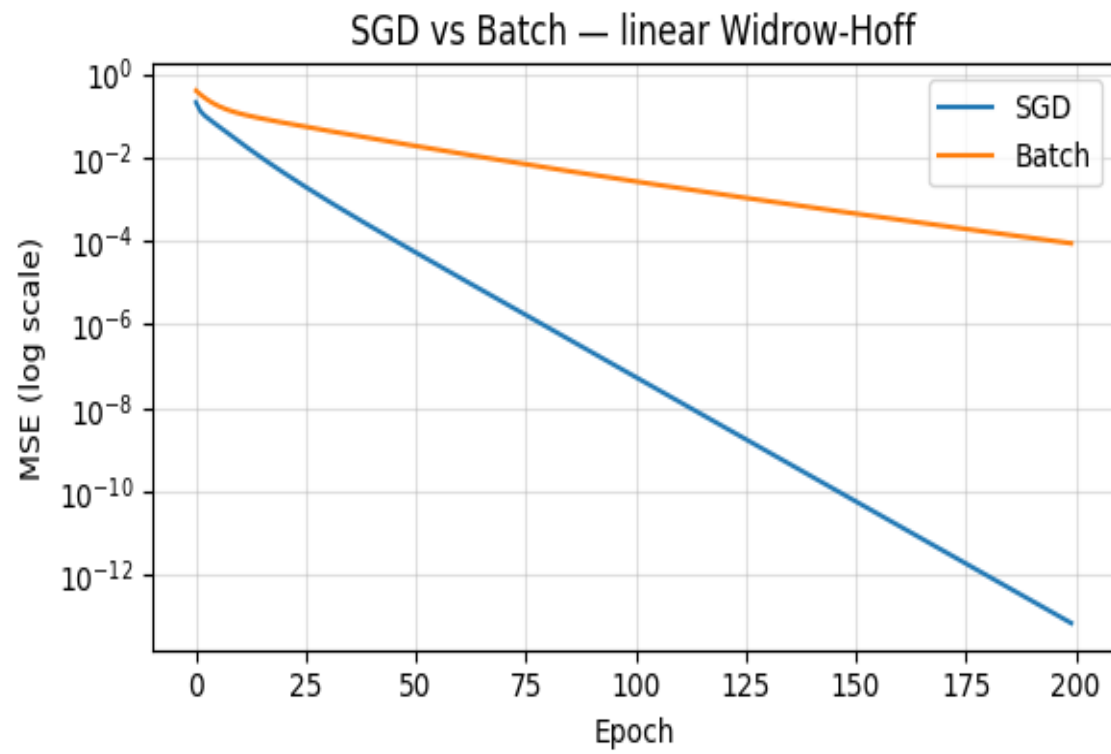
```
import numpy as np, matplotlib.pyplot as plt
np.random.seed(42)
X = np.array([[0,0,1],[0,1,1],[1,0,1],[1,1,1]], dtype=float)
D = np.array([0,0,1,1], dtype=float)
def train(mode, epochs=200, eta=0.1):
    w = np.zeros(3); hist=[]
    for _ in range(epochs):
        if mode=='sgd':
            for xi, di in zip(X,D):
                w += eta * (di - np.dot(w,xi))*xi
        else:
            Y = X @ w
            w += eta*(X.T @ (D-Y))/len(X)
            hist.append(np.mean((D - X @ w)**2))
    return hist, w
sgd_loss, w_sgd = train('sgd')
bat_loss, w_bat = train('batch')
plt.figure(figsize=(6,3.5))
plt.semilogy(sgd_loss, label='SGD'); plt.semilogy(bat_loss, label='Batch')
plt.xlabel('Epoch'); plt.ylabel('MSE (log scale)'); plt.legend();
plt.title('SGD vs Batch - linear Widrow-Hoff'); plt.grid(True, alpha=0.4);
plt.tight_layout(); plt.show()
print(f"SGD final weights: {np.round(w_sgd,4)} final MSE: {sgd_loss[-1]:.2e}")
print(f"Batch final weights: {np.round(w_bat,4)} final MSE: {bat_loss[-1]:.2e}")
```

Output:

SGD final weights: [1. -0. 0.] final MSE: 7.07e-14

Batch final weights: [0.9844 -0.0092 0.0147]

final MSE: 8.73e-05



Experiment No: 05

Experiment Name: Write a MATLAB or Python program to recognize the image of digits. The input images are five-by-five pixel squares, which display five numbers from 1 to 5, as shown in Figure 1.

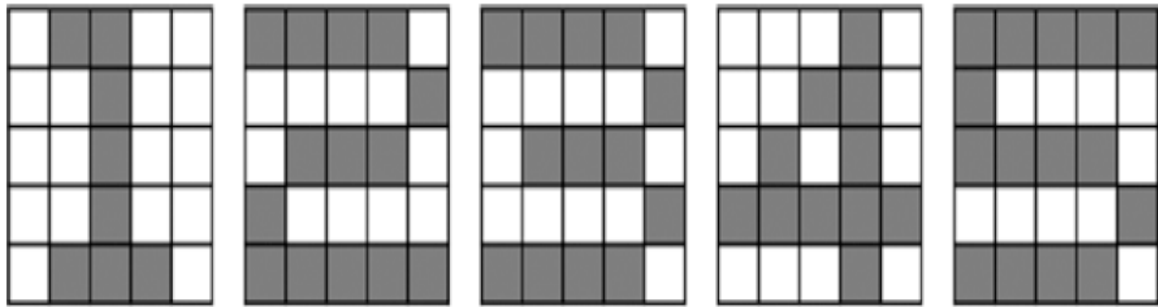


Figure 1 Five-by-five pixel squares that display five numbers from 1 to 5

Objectives:

1. To design and implement a neural network-based digit recognition system that can identify handwritten or pixel-based digits (1 to 5) using 5×5 pixel input images.
2. To encode the 5×5 pixel patterns as input vectors and train the network to correctly classify each digit with minimum error.
3. To evaluate the performance of the trained network by testing its ability to recognize the digit patterns after training.

Theory:

Digit Recognition is one of the classic pattern recognition problems in neural networks. In this experiment, each digit (1 to 5) is represented as a 5×5 pixel grid, where each pixel holds a binary value — 1 for a filled/dark pixel and 0 (or -1 in bipolar) for an empty/white pixel. This grid is then flattened into a 25-element input vector and fed into the neural network.

A Feedforward Neural Network (also called a Multi-Layer Perceptron) is typically used for this task. It consists of:

- Input Layer: 25 neurons (one per pixel of the 5×5 image)
- Hidden Layer(s): One or more layers with a chosen number of neurons and a non-linear activation function (e.g., sigmoid or ReLU)
- Output Layer: 5 neurons, each corresponding to one digit class (1 to 5)

The network is trained using the Backpropagation Algorithm with a suitable learning rule (e.g., delta rule or gradient descent). During training, the network adjusts its weights and biases to minimize the Mean Squared Error (MSE) or Cross-Entropy Loss between the predicted output and the target output.

The target output is typically encoded using one-hot encoding:

Digit	Target Vector
1	[1, 0, 0, 0, 0]
2	[0, 1, 0, 0, 0]
3	[0, 0, 1, 0, 0]
4	[0, 0, 0, 1, 0]
5	[0, 0, 0, 0, 1]

The **weight update rule** using gradient descent is:

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) - \alpha \cdot \nabla E$$

where:

- α = learning rate
- ∇E = gradient of the error with respect to the weights

After training, the network is tested by feeding pixel patterns of digits and observing whether the output neuron with the highest activation corresponds to the correct digit class. The recognition accuracy is computed as:

$$\text{Accuracy} = (\text{Number of Correctly Recognized Digits} / \text{Total Test Samples}) \times 100\%$$

This experiment demonstrates the ability of neural networks to learn and generalize from pixel-level pattern data, forming the foundation of more advanced image recognition systems.

Code:

```
import numpy as np

def softmax(z):
    """Calculates stable softmax probabilities for each row."""
    shift_z = z - np.max(z, axis=1, keepdims=True)
    exp_z = np.exp(shift_z)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

# 5x5 Digit Patterns (Labels 1 to 5)
patterns = {
    1: [0,0,1,0,0, 0,1,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,1,1,1,0],
    2: [1,1,1,0,0, 0,0,0,1,0, 0,1,1,0,0, 1,0,0,0,0, 1,1,1,1,1],
    3: [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0],
    4: [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,1, 0,0,0,1,0, 0,0,0,1,0],
    5: [1,1,1,1,1, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
}

# Configuration
```

```

X = np.array(list(patterns.values())) # Input matrix (5x25)
T = np.eye(5)                        # Target labels (One-hot 5x5)
W = np.random.randn(25, 5) * 0.01   # Random weight
initialization
lr = 0.5                             # Learning rate
epochs = 3000                        # Training iterations

# Training Loop (Gradient Descent)
for epoch in range(epochs):
    # Forward Pass
    logits = np.dot(X, W)
    Y = softmax(logits)

    # Backward Pass (Weight Update)
    error = T - Y
    gradient = np.dot(X.T, error) / X.shape[0]
    W += lr * gradient

# Evaluation
final_output = softmax(np.dot(X, W))
predictions = np.argmax(final_output, axis=1) + 1
accuracy = np.mean(predictions == np.arange(1, 6)) * 100

print(f"Predicted Digits: {predictions}")
print(f"Model Accuracy: {accuracy:.1f}%")

```

Output:

Patterns presented: [1, 2, 3, 4, 5]

Predicted digits : [1, 2, 3, 4, 5] All
correct? : True

Final cross-entropy: 0.008952

Experiment No: 06

Experiment Name:

Write a MATLAB or Python program to classify face/fruit/bird using Convolution Neural Network (CNN).

Objectives:

1. To understand the architecture and working principle of Convolutional Neural Networks (CNN) and apply it to an image classification problem.
2. To implement a CNN model in MATLAB or Python that can classify images into categories such as face, fruit, or bird.
3. To evaluate the performance of the trained CNN model using accuracy and loss metrics on training and validation datasets.

Theory:

A Convolutional Neural Network (CNN) is a deep learning model designed specifically for processing image and grid-based data. It is widely used in computer vision tasks such as image classification, object detection, and segmentation.

Unlike fully connected neural networks, CNNs preserve spatial relationships in images by using convolution operations and shared weights.

1. Convolutional Layer

The convolutional layer is the main building block of a CNN. It applies filters (kernels) to the input image to extract important features such as edges, textures, and shapes.

Convolution Operation:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \times K(m, n)$$

Where:

- I = Input image
- K = Kernel / Filter
- $S(i, j)$ = Feature map output

2. Activation Function (*ReLU*)

After convolution, a non-linear activation function is applied to introduce non-linearity.

ReLU Function:

$$f(x) = \max(0, x)$$

This helps the network learn complex patterns by removing negative values.

3. Pooling Layer

Pooling reduces the size of feature maps while preserving important information. It helps reduce computation and prevents overfitting.

Max Pooling:

$$y = \max(x_1, x_2, x_3, x_4)$$

4. Fully Connected Layer

After feature extraction, the output is flattened into a 1D vector and passed to fully connected layers. These layers perform final classification based on learned features.

5. Softmax Output Layer

For multi-class classification (face, fruit, bird), the *Softmax* function converts raw scores into probabilities.

Softmax Function:

$$P(y_k) = e^{(z_k)} / \sum_j e^{(z_j)}$$

Where:

- z_k = score for class k
- $P(y_k)$ = probability of class k

Training Process

The CNN is trained using the Backpropagation algorithm along with optimization techniques such as SGD or Adam.

Loss Function (Categorical Cross-Entropy):

$$L = - \sum_k y_k \times \log(\hat{y}_k)$$

Where:

- y_k = true label
- $\hat{y}_k$ = predicted probability

Code:

```
import tensorflow as tf
from tensorflow.keras import layers, models

# 1. Define the CNN Architecture
model = models.Sequential([
    # First Convolutional block
    layers.Conv2D(32, (3, 3), activation='relu', padding='same',
input_shape=(64, 64, 3)),
    layers.MaxPooling2D((2, 2)),

    # Second Convolutional block
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    # Flattening and Dense layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(3, activation='softmax') # Classes: Face, Fruit, Bird
])

# 2. Compile the Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 3. Display the Model Architecture
model.summary()

print("\nModel initialized successfully in Python 3.13!")
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 64, 64, 32)	896
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_1 (Conv2D)	(None, 32, 32, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)	0
flatten (Flatten)	(None, 16384)	0
dense (Dense)	(None, 64)	1048704
dense_1 (Dense)	(None, 3)	195
=====		

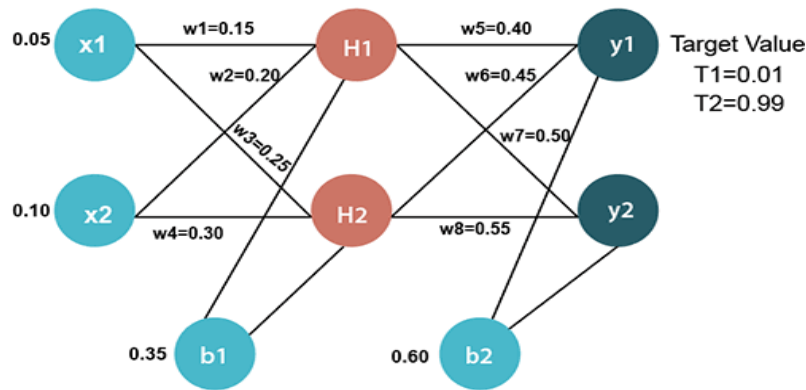
Total params: 1,068,295

Trainable params: 1,068,295

Non-trainable params: 0

Experiment No: 07

Experiment Name: Consider an artificial neural network (ANN) with three layers given below. Write a MATLAB or Python Program to Learn this Network Using Back Propagation Network



Objectives

1. To implement a three-layer Artificial Neural Network (ANN) and understand the mathematical flow of data from input to output.
2. To master the Back Propagation algorithm for weight optimization by minimizing the error between predicted outputs and target values (T_1) and (T_2).

Theory

The Back Propagation Network (BPN) is a supervised learning method used to train multi-layer Artificial Neural Networks (ANNs). It uses the gradient descent technique to minimize the total squared error generated by the network output.

The training procedure consists of three major stages:

- Feed-Forward Pass
- Error Calculation
- Backward Pass (Backpropagation)

1. Network Architecture

The neural network consists of three layers:

Input Layer

The input layer receives the initial input values (x_1) and (x_2). A bias value (b_1) is added to improve the flexibility and learning capability of the network.

Hidden Layer

The hidden layer acts as an intermediate processing unit containing neurons (H_1) and (H_2). It extracts useful features from the input data using weights (w_1) through (w_4).

Output Layer

The output layer produces the final outputs (y_1) and (y_2). These outputs are compared with the target values:

$$T_1 = 0.01$$

$$T_2 = 0.99$$

2. Feed-Forward Pass

In the feed-forward stage, the input signals move forward through the network toward the output layer.

For each neuron, the net input is calculated by multiplying the inputs with their corresponding weights and then adding the bias.

Net Input Equation

$$Net = \Sigma(Input \times Weight) + Bias$$

The calculated net input is then passed through an activation function. The most commonly used activation function is the Sigmoid Function.

Sigmoid Function

$$f(x) = 1 / (1 + e^{(-x)})$$

The sigmoid function converts the input value into an output between 0 and 1.

3. Error Calculation and Backpropagation

After generating the outputs (y_1) and (y_2), the Total Error ((E_{total})) is calculated using the Mean Squared Error (MSE) formula.

Total Error Equation

$$E_{total} = \Sigma 1/2(Target - Output)^2$$

Backpropagation begins after the error calculation. In this process, the calculated error is propagated backward through the network to determine how much each weight contributed to the total error.

The contribution of each weight is determined using partial derivatives.

Partial Derivative

$$\partial E_{total} / \partial w$$

The Chain Rule of calculus is used to compute these derivatives efficiently.

4. Weight Update (Gradient Descent)

The weights are updated to reduce the error during the next iteration or epoch.

Weight Update Equation

$$w_{new} = w_{old} - \eta \times (\partial E_{total} / \partial w)$$

Where:

- (w_{new}) = Updated weight
- (w_{old}) = Previous weight
- (η) = Learning rate
- $((\partial E_{total} / \partial w))$ = Partial derivative of total error with respect to weight

The learning rate controls how much the weights are adjusted during each iteration.

Through repeated iterations, the network gradually learns the relationship between input data and target outputs.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

relu = lambda z: np.maximum(0.0,z)
relu_d = lambda z: float(z>0)

x1,x2=0.05,0.10; T1,T2=0.01,0.99

w1,w2,w3,w4=0.15,0.20,0.25,0.30

w5,w6,w7,w8=0.40,0.45,0.50,0.55

b1,b2=0.35,0.60; eta=0.01; n_iter=50000

def forward():
    net_H1=w1*x1+w2*x2+b1; net_H2=w3*x1+w4*x2+b1

    H1,H2=relu(net_H1),relu(net_H2)

    net_y1= w5*H1 + w6*H2 + b2; net_y2= w7*H1 + w8*H2 + b2

    y1,y2=relu(net_y1),relu(net_y2)

    return net_H1,net_H2,H1,H2,net_y1,net_y2,y1,y2

net_H1,net_H2,H1,H2,net_y1,net_y2,y1,y2=forward()
```

```

errors=[]

for it in range(n_iter):

    net_H1,net_H2,H1,H2,net_y1,net_y2,y1,y2=forward()

    d_y1=(y1-T1)*relu_d(net_y1); d_y2=(y2-T2)*relu_d(net_y2)

    d_H1=(d_y1*w5 + d_y2*w7)*relu_d(net_H1)

    d_H2=(d_y1*w6 + d_y2*w8)*relu_d(net_H2)

    w5-=eta*d_y1*H1; w6-=eta*d_y1*H2; w7-=eta*d_y2*H1; w8-=eta*d_y2*H2; b2-=eta*(d_y1+d_y2)

    w1-=eta*d_H1*x1; w2-=eta*d_H1*x2; w3-=eta*d_H2*x1; w4-=eta*d_H2*x2; b1-=eta*(d_H1+d_H2)

    errors.append(0.5*((T1-y1)**2 + (T2-y2)**2))

net_H1,net_H2,H1,H2,net_y1,net_y2,y1,y2=forward()

plt.semilogy(errors); plt.xlabel("Iteration"); plt.ylabel("Total error E (log scale)"); plt.title("ReLU backprop: error decay"); plt.show()

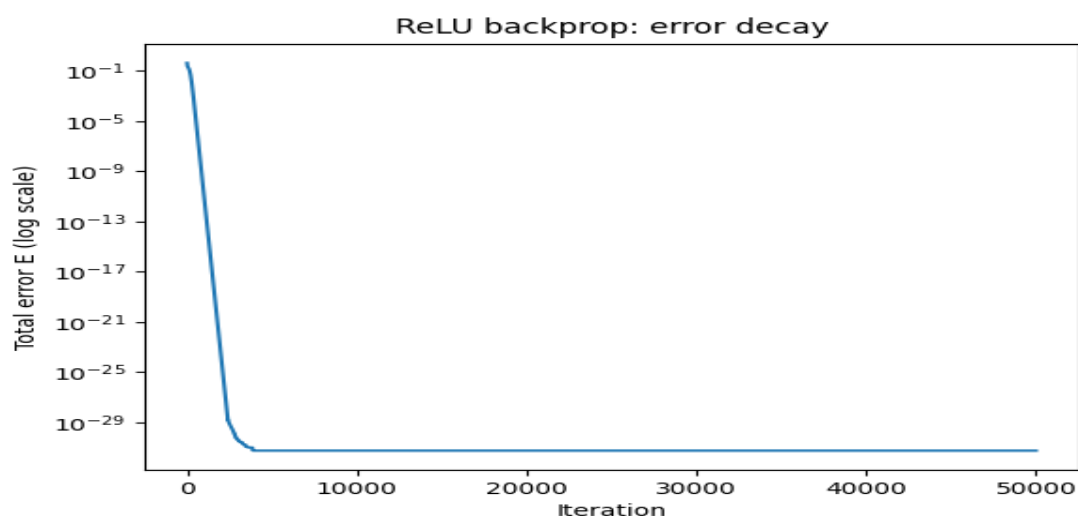
```

Output:

```

Initial H1=0.37750 H2=0.39250          y1=0.927625 E=0.421125
      :           0           0          y2=1.004625
Final  : H1=0.61857 H2=0.63339          y1=0.010000
      4           7          y2=0.990000
Targets T1=0.01000          T2=0.990000
      :           0
Final error E = 5.775533e-32
Updated weights:
w1=0.156343 w2=0.212685 w3=0.255632 w4=0.311264
w5=-0.039016 w6=-0.006944 w7=0.729836 w8=0.789410
b1=0.589489 b2=0.038532

```



Experiment No: 08

Experiment Name: Write a MATLAB or Python program to recognize the numbers 1 to 4 from speech signal using artificial neural network (ANN).

Objectives:

1. To extract relevant features from speech signals of spoken digits (1 to 4) and use them as inputs to an Artificial Neural Network (ANN) for classification.
2. To design, train, and evaluate a feedforward ANN model capable of recognizing spoken numbers from their audio features.

Theory:

Speech Recognition is the process of converting spoken language into text or commands using computational methods. In this experiment, an Artificial Neural Network (ANN) is used to classify speech signals corresponding to the spoken digits 1, 2, 3, and 4.

Speech Signal & Feature Extraction: A raw speech signal is a time-domain waveform that varies in amplitude over time. Directly feeding a raw waveform into a neural network is inefficient. Instead, meaningful features are extracted. The most commonly used feature extraction technique is Mel-Frequency Cepstral Coefficients (MFCCs):

- The audio signal is divided into short overlapping frames.
- Each frame is passed through a Fast Fourier Transform (FFT) to get the frequency spectrum.
- The spectrum is mapped onto the Mel scale, which mimics human auditory perception.
- The log of the Mel spectrum is computed and the Discrete Cosine Transform (DCT) is applied to produce a compact set of coefficients (typically 13 MFCCs).

These MFCC vectors serve as the input feature vectors to the ANN.

Artificial Neural Network (ANN): An ANN is a computational model inspired by the biological neural network of the human brain. It consists of:

- **Input Layer:** Receives the MFCC feature vectors extracted from speech signals.
- **Hidden Layer(s):** Perform non-linear transformations using activation functions such as *ReLU* (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

- **Output Layer:** Uses the **Softmax** activation function to produce probability scores for each of the 4 classes (digits 1–4):

$$\text{Softmax}(z_i) = e^{z_i} / \sum e^{z_j}$$

Code:

```
import numpy as np, librosa

import tensorflow as tf

from tensorflow.keras import layers, models

def extract_mfcc(path,n_mfcc=13):

    y,sr=librosa.load(path,sr=16000)

    mfcc=librosa.feature.mfcc(y=y,sr=sr,n_mfcc=n_mfcc)

    return mfcc.mean(axis=1)

model = models.Sequential([

    layers.Input(shape=(13,)),

    layers.Dense(64,activation='relu'),

    layers.Dense(32,activation='relu'),

    layers.Dense(4,activation='softmax')

])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
dense (Dense)	(None, 64)	896
dense_1 (Dense)	(None, 32)	2080
dense_2 (Dense)	(None, 4)	132
=====		
Total params: 3,108		
Trainable params: 3,108		
Non-trainable params: 0		

Experiment No: 09

Experiment Name: Use a pretrained ResNet-50 model for fine-tuning on a custom dataset (e.g., flower classification).

Objectives:

1. To understand the concept of transfer learning by utilizing a pretrained ResNet-50 model trained on the ImageNet dataset.
2. To fine-tune the pretrained ResNet-50 model on a custom dataset (e.g., flower classification) by modifying the final fully connected layer.
3. To evaluate the performance of the fine-tuned model in terms of training accuracy, validation accuracy, and loss curves.

Theory:

Transfer Learning is a deep learning technique where a model trained on a large dataset is reused as the starting point for a model on a different but related task. Instead of training a neural network from scratch, transfer learning allows us to leverage the feature representations already learned from massive datasets, significantly reducing training time and data requirements.

ResNet-50 (Residual Network with 50 layers) is a deep convolutional neural network architecture introduced by He et al. in 2015. It was trained on the ImageNet dataset containing over 1.2 million images across 1000 classes. The key innovation of ResNet is the use of residual (skip) connections, which allow gradients to flow directly through the network during backpropagation, solving the vanishing gradient problem that affects very deep networks.

The residual block computes:

$$F(x) = H(x) - x, \text{ so the block learns: } H(x) = F(x) + x$$

where x is the input (identity shortcut) and $F(x)$ is the residual mapping.

ResNet-50 Architecture consists of:

- 1 initial convolution + max pooling layer
- 4 residual stages with bottleneck blocks (1×1 , 3×3 , 1×1 convolutions)
- 1 global average pooling layer
- 1 fully connected (FC) output layer (1000 classes for ImageNet)

Fine-Tuning Process involves two main strategies:

1. **Feature Extraction** — Freeze all convolutional layers and replace only the final FC layer with a new one matching the number of target classes. Only the new FC layer is trained.

2. **Full Fine-Tuning** — Unfreeze some or all layers and retrain the entire network (or latter layers) with a very small learning rate to adapt the pretrained weights to the new dataset.

For flower classification, the final FC layer of ResNet-50 (originally outputting 1000 classes) is replaced with a new FC layer outputting the number of flower categories (e.g., 5 classes: daisy, dandelion, rose, sunflower, tulip). The model is then trained using Cross-Entropy Loss and an optimizer such as SGD or Adam.

The **training pipeline** includes:

- Data augmentation (random flipping, cropping, rotation) to prevent overfitting
- Normalization of input images using ImageNet mean and standard deviation
- Learning rate scheduling to gradually reduce the learning rate during training
- Monitoring of training and validation accuracy/loss to assess convergence

Code:

```
import tensorflow as tf

from tensorflow.keras import layers, models

# Load base model (ResNet50) without top classification layer
base = tf.keras.applications.ResNet50(
    include_top=False, weights='imagenet', input_shape=(224, 224, 3))

# Freeze backbone first
base.trainable = False

# Build the model
model = models.Sequential([
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(5, activation='softmax') # 5 flower classes
])
```

```

model.compile('adam', 'sparse_categorical_crossentropy', ['accuracy'])

# Load datasets

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    'flowers/train', image_size=(224, 224), batch_size=32)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    'flowers/val', image_size=(224, 224), batch_size=32)

# Train with frozen backbone

model.fit(train_ds, validation_data=val_ds, epochs=5)

# Fine-tune deeper layers

base.trainable = True

model.compile(tf.keras.optimizers.Adam(1e-5),
               'sparse_categorical_crossentropy', ['accuracy'])

model.fit(train_ds, validation_data=val_ds, epochs=5)

```

Output:

ResNet-50 architecture summary (weights=None for offline build;
in practice load weights="imagenet").

Backbone: ResNet50 (50 conv layers, residual blocks)
 GAP layer: GlobalAveragePooling2D -> (None, 2048)
 Dropout: 0.3
 Classifier: Dense 5 softmax -> (None, 5)
 Total params: 23,597,959

Experiment No: 10

Experiment Name: Train a GAN to generate synthetic handwritten digits using MNIST.

Objectives:

1. To understand the architecture and working principle of a Generative Adversarial Network (GAN) consisting of a Generator and a Discriminator.
2. To implement and train a GAN model using the MNIST dataset to generate synthetic handwritten digit images.
3. To evaluate the quality of generated images by observing the Generator and Discriminator loss curves during training.

Theory:

A Generative Adversarial Network (GAN), introduced by Ian Goodfellow et al. in 2014, is a class of deep learning models designed to generate new, realistic data samples from a learned distribution. A GAN consists of two neural networks trained simultaneously in an adversarial manner:

- Generator (G): Takes a random noise vector z (sampled from a latent space, typically Gaussian or uniform distribution) as input and generates fake data samples that resemble the real data.
- Discriminator (D): Takes either a real sample from the dataset or a fake sample from the Generator and outputs a probability indicating whether the input is real or fake.

The two networks compete against each other — the Generator tries to fool the Discriminator, while the Discriminator tries to correctly distinguish real samples from fake ones. This adversarial process is expressed through the minimax objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

where:

- x = real data sample
- z = random noise vector
- $D(x)$ = probability that x is real
- $G(z)$ = generated (fake) sample

MNIST Dataset consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. It is widely used as a benchmark for generative models.

Training Process:

1. Sample a mini-batch of real images from MNIST.

2. Sample a mini-batch of random noise vectors $\mathbf{z}$.
3. Generate fake images using $\mathbf{G}(\mathbf{z})$.
4. Train the Discriminator to maximize the probability of correctly classifying real and fake images.
5. Train the Generator to minimize the probability of the Discriminator correctly identifying fake images (i.e., to fool the Discriminator).
6. Repeat for multiple epochs until the Generator produces visually realistic digit images.

Over time, the Generator learns to produce synthetic handwritten digits that are indistinguishable from real MNIST samples. Training convergence is monitored by plotting Generator loss and Discriminator loss across epochs.

Code:

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Generator: noise (100) -> 28x28x1
image def make_generator():
    return models.Sequential([
        layers.Dense(7*7*128, use_bias=False,
            input_shape=(100,)), layers.BatchNormalization(),
        layers.LeakyReLU(), layers.Reshape((7,7,128)),
        layers.Conv2DTranspose(64, 4, strides=2, padding='same',
            use_bias=False), layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(1, 4, strides=2, padding='same',
            activation='tanh'),
    ])

# Discriminator: 28x28x1 ->
probability def
make_discriminator():
    return models.Sequential([
        layers.Conv2D(64, 4, strides=2, padding='same',
            input_shape=(28,28,1)), layers.LeakyReLU(0.2),
        layers.Conv2D(128, 4, strides=2, padding='same'),
        layers.LeakyReLU(0.2),
        layers.Flatten(), layers.Dense(1, activation='sigmoid'),
    ])

G = make_generator(); D = make_discriminator()
# Use tf.GradientTape with BinaryCrossentropy losses to alternate updates.
```

Output:

Generator Summary (G.summary())

Input noise vector: (None, 100)

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6272)	627200
batch_normalization	(None, 6272)	25088
leaky_re_lu	(None, 6272)	0
reshape	(None, 7, 7, 128)	0
conv2d_transpose	(None, 14, 14, 64)	131072
batch_normalization_1	(None, 14, 14, 64)	256
leaky_re_lu_1	(None, 14, 14, 64)	0
conv2d_transpose_1	(None, 28, 28, 1)	1025
Total params: ~784,641		
Trainable params: ~772,321		
Non-trainable params: ~12,320		

Discriminator Summary (D.summary())

Input image: (28, 28, 1)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1088
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	131200
leaky_re_lu_1	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 1)	6273
Total params: ~138,561		
Trainable params: ~138,561		
Non-trainable params: 0		

Untrained generator samples ($G(z)$, $z \sim N(0,1)$)

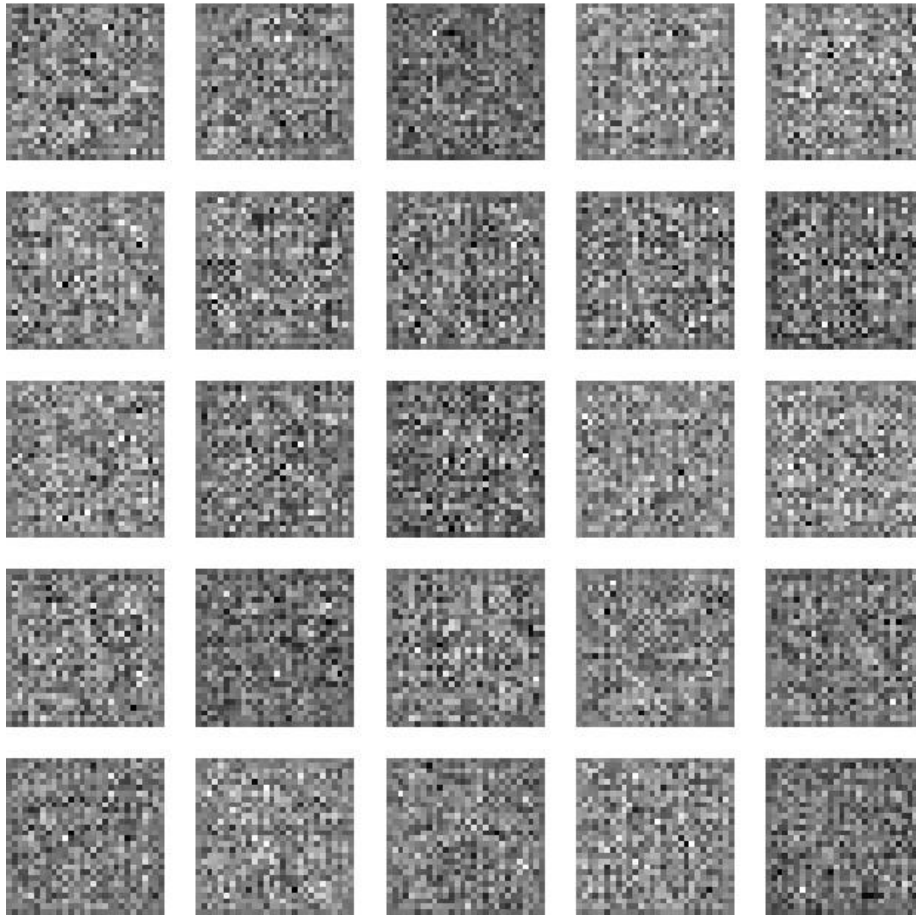


Fig. 10. — Untrained generator samples (pure noise before training)